



---

# PROJE TASARIM RAPORU

---

**Ders:** Yazılım Mühendisliği  
**Bölüm/Şube:** Bilgisayar Mühendisliği  
**Grup Adı:** Grup 2  
**Proje Adı:** Banka Yönetim Sistemi  
**Teslim Tarihi:** 03.05.2026

**Grup Üyeleri:**  
Abdullah Arpacı 23100011033  
Yemliha Can Yastı 24100011016  
Duran Can Çetin 24100011024  
Yaşar Samet Üçoğlan 24100011032  
Muhammed Yusuf Erzi 24100011052

## İçindekiler

|                                                      |    |
|------------------------------------------------------|----|
| <b>1. Giriş</b>                                      | 3  |
| <b>1.1. Sistemin Amacı</b>                           | 3  |
| <b>1.2. Tasarım Hedefleri</b>                        | 3  |
| <b>1.2.1. Güvenlik (Security)</b>                    | 3  |
| <b>1.2.2. Kullanılabilirlik</b>                      | 4  |
| <b>1.2.3. Veri Bütünlüğü ve Güvenilirlik</b>         | 4  |
| <b>1.2.4. Bakım Yapılabilirlik</b>                   | 4  |
| <b>1.2.5. Performans</b>                             | 4  |
| <b>2. Üst Düzey Yazılım Mimarisi</b>                 | 4  |
| <b>2.1. Alt Sistem Ayrıştırması</b>                  | 5  |
| <b>2.2. Donanım/Yazılım Eşlemesi</b>                 | 6  |
| <b>2.3. Kalıcı Veri Yönetimi</b>                     | 7  |
| <b>2.4. Erişim Kontrolü ve Güvenlik</b>              | 9  |
| <b>2.5. Küresel Kontrol Akışı</b>                    | 10 |
| <b>2.5.1. Merkezi İş Mantığı (SistemKontrolcüsü)</b> | 10 |
| <b>2.5.2. İşlem Yaşam Döngüsü</b>                    | 11 |
| <b>2.5.3. Güvenlik ve Hata Yönetimi</b>              | 12 |
| <b>2.5.4. Eşzamanlılık ve Veri Tutarlılığı</b>       | 12 |
| <b>2.6. Sınır Koşulları</b>                          | 12 |
| <b>2.6.1. Başlatma</b>                               | 12 |
| <b>2.6.2. Sonlandırma</b>                            | 13 |
| <b>2.6.3. Hata Durumları</b>                         | 13 |
| <b>2.6.4. Belirsiz Durumlar</b>                      | 14 |
| <b>3. Alt Düzey Tasarım</b>                          | 14 |
| <b>3.1. Nesne Tasarım Kararları</b>                  | 14 |
| <b>3.2. Son Nesne Tasarımı</b>                       | 15 |
| <b>3.2.1. Temel Varlık Sınıfları</b>                 | 15 |
| <b>3.2.2. Destek Varlık Sınıfları</b>                | 18 |
| <b>3.2.3. Servis ve Kontrol Katmanı Sınıfları</b>    | 19 |
| <b>3.3. Paketler / Katmanlar</b>                     | 1  |
| <b>3.3.1. Dış Paketler</b>                           | 1  |
| <b>3.3.2. İç Paketler</b>                            | 1  |
| <b>3.4. Sınıf Arayüzleri</b>                         | 2  |
| <b>3.5. Tasarım Desenleri</b>                        | 5  |
| <b>Sözlük (Glossary)</b>                             | 6  |

## 1. Giriş

Bu bölümde geliştirilen Banka Yönetim Sistemi'nin amacı ve tasarım sürecinde dikkate alınan temel hedefler açıklanmaktadır. Sistem, masaüstü ortamında çalışan ve temel bankacılık işlemlerini simüle eden bir yazılım olarak tasarlanmış olup, kullanıcıların finansal işlemleri güvenli, hızlı ve kullanıcı dostu bir arayüz üzerinden gerçekleştirebilmesini amaçlamaktadır.

### 1.1. Sistemin Amacı

Banka Yönetim Sistemi'nin temel amacı, kullanıcıların hesap yönetimi, para transferi, bakiye görüntüleme ve işlem geçmişi takibi gibi bankacılık işlemlerini gerçekleştirebildiği bir simülasyon ortamı sunmaktır. Sistem, gerçek bankacılık uygulamalarının temel işleyiş mantığını modelleyerek kullanıcıların bu süreçleri daha iyi anlamasını hedeflemektedir. Ayrıca farklı kullanıcı rollerine göre yetkilendirme yaparak, her kullanıcının yalnızca kendi yetki alanındaki işlemleri gerçekleştirebilmesi sağlanmaktadır. Bu sayede sistem hem eğitim amaçlı kullanılabilir hem de güvenli işlem yönetimini örnekleyen bir yazılım olarak işlev görmektedir.

### 1.2. Tasarım Hedefleri

Sistemin geliştirilmesi sırasında yalnızca işlevsel gereksinimler değil, aynı zamanda kaliteyi artıran işlevsel olmayan gereksinimler de dikkate alınmıştır. Bu kapsamda güvenlik, kullanılabilirlik, veri bütünlüğü, bakım yapılabilirlik ve performans gibi temel tasarım hedefleri belirlenmiş ve sistem bu hedefler doğrultusunda yapılandırılmıştır.

#### 1.2.1. Güvenlik

Güvenlik, bankacılık sistemleri için en kritik tasarım hedeflerinden biridir. Sistem içerisinde kullanıcı bilgileri, hesap bakiyeleri ve işlem kayıtları gibi hassas veriler bulunduğundan, bu verilerin yetkisiz erişimlere karşı korunması gerekmektedir. Bu doğrultuda sistemde kullanıcı doğrulama mekanizmaları uygulanmış, kullanıcı şifreleri açık metin olarak saklanmak yerine güvenli hash algoritmaları ile korunmuştur. Ayrıca rol tabanlı yetkilendirme sayesinde kullanıcıların yalnızca izin verilen işlemleri gerçekleştirebilmesi sağlanmış ve sistemde gerçekleşen kritik işlemler kayıt altına alınarak olası güvenlik ihlallerine karşı izlenebilirlik artırılmıştır.

### **1.2.2. Kullanılabilirlik**

Sistemin kullanıcılar tarafından kolay anlaşılabilir ve rahat kullanılabilir olması önemli bir tasarım hedefidir. Bu nedenle arayüz tasarımında sadelik ve anlaşılabilirlik ön planda tutulmuştur. Qt framework kullanılarak geliştirilen grafik kullanıcı arayüzü sayesinde kullanıcılar işlemlerini karmaşık adımlar olmadan gerçekleştirebilmektedir. Ayrıca sistem, kullanıcı hatalarını minimize etmek amacıyla yönlendirici mesajlar ve uyarılar içermekte olup, işlem süreçleri açık ve anlaşılır bir şekilde sunulmaktadır.

### **1.2.3. Veri Bütünlüğü ve Güvenilirlik**

Finansal işlemlerde veri kaybı veya tutarsızlık oluşması ciddi problemlere yol açabileceğinden, sistemin güvenilir ve tutarlı çalışması büyük önem taşımaktadır. Bu kapsamda veriler SQLite veritabanı üzerinde ACID prensiplerine uygun olarak saklanmakta ve tüm işlemler kontrollü bir şekilde gerçekleştirilmektedir. İşlem sırasında oluşabilecek hatalara karşı gerekli kontroller yapılmakta ve sistemin tutarlı bir veri yapısını koruması sağlanmaktadır. Bu sayede kullanıcı işlemleri güvenilir bir şekilde kayıt altına alınmakta ve sistem doğruluğu korunmaktadır.

### **1.2.4. Bakım Yapılabilirlik**

Sistemin uzun vadede geliştirilebilir ve sürdürülebilir olması amacıyla bakım yapılabilirlik önemli bir tasarım hedefi olarak belirlenmiştir. Bu doğrultuda sistem katmanlı mimari prensiplerine uygun olarak tasarlanmış ve nesne yönelimli programlama yaklaşımı benimsenmiştir. Modüler yapı sayesinde sistem bileşenleri birbirinden bağımsız şekilde geliştirilebilmekte ve gerektiğinde kolayca güncellenebilmektedir. Bu yaklaşım, hem hata ayıklama sürecini kolaylaştırmakta hem de yeni özelliklerin sisteme entegrasyonunu hızlandırmaktadır.

### **1.2.5. Performans**

Sistemin farklı donanım seviyelerine sahip cihazlarda akıcı bir şekilde çalışabilmesi için performans önemli bir kriter olarak ele alınmıştır. Bu amaçla uygulama C++ programlama dili kullanılarak geliştirilmiş ve düşük kaynak tüketimi hedeflenmiştir. Ayrıca veritabanı işlemlerinde yerel SQLite kullanılarak hızlı veri erişimi sağlanmıştır. Gereksiz işlem yüklerinden kaçınılarak sistemin genel performansı optimize edilmiş ve kullanıcı deneyiminin kesintisiz olması hedeflenmiştir.

## **2. Üst Düzey Yazılım Mimarisi**

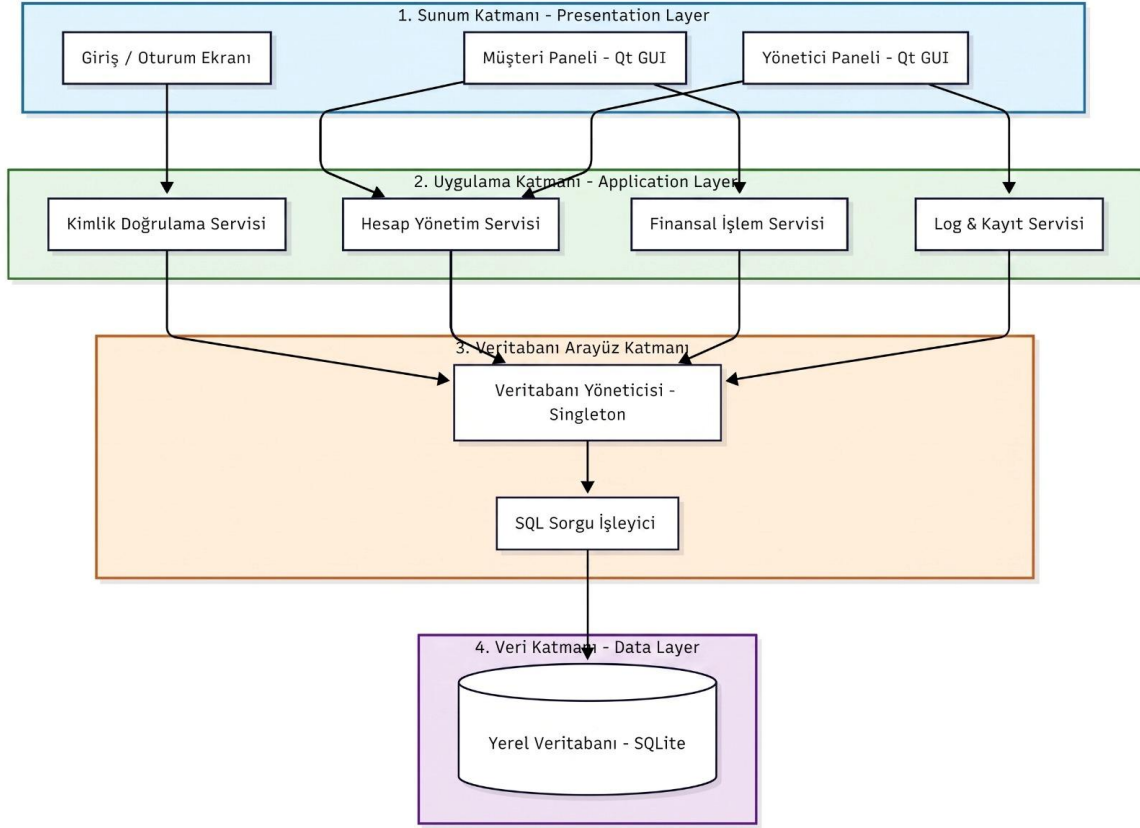
Bu bölümde, C++ ve Qt Framework teknolojileri kullanılarak masaüstü ortamında geliştirilen Banka Yönetim Sistemi'nin teknik iskeleti ve temel mimari yapısı ele

alınmaktadır. Sistem kapsamında; katmanlı mimariye dayalı alt sistem ayrıştırması, donanım ve yazılım bileşenlerinin eşleştirilmesi, yerel veritabanı üzerinde kalıcı veri yönetimi, kullanıcı yetkilendirme mekanizmaları ile erişim kontrolü ve güvenlik önlemleri incelenmektedir.

## 2.1. Alt Sistem Ayrıştırması

Bankacılık Sistemi, sürdürülebilirlik, ölçeklenebilirlik ve güvenlik hedefleri doğrultusunda katmanlı mimari (layered architecture) prensiplerine uygun olarak tasarlanmıştır. Sistem, nesne yönelimli programlama (OOP) standartları gözetilerek, birbirleriyle düşük bağımlılıkla çalışan dört temel alt sisteme ayrılmıştır:

- **Sunum Katmanı (Presentation Layer):** Qt framework'ü kullanılarak geliştirilen ve kullanıcının sistemle doğrudan etkileşime girdiği grafik kullanıcı arayüzünü (GUI) barındıran katmandır. Müşteri ve Yönetici (Admin) olmak üzere iki farklı aktör için özelleştirilmiş paneller bu katmanda yer alır. Kullanıcıdan alınan girdi verileri doğrulanmak ve işlenmek üzere uygulama katmanına iletilir.
- **Uygulama Katmanı (Application Layer):** C++ dili ile kodlanan ve sistemin temel iş mantığının yürütüldüğü merkezi katmandır. Kullanıcı doğrulama, bakiye güncelleme, hesap açma/kapatma ve para transferi gibi temel finansal operasyonlar bu katmanda bulunan nesneler aracılığıyla gerçekleştirilir.
- **Veritabanı Arayüz Katmanı (Database Interface Layer):** Uygulama katmanı ile veri depolama birimleri arasındaki alışverişi soyutlayan ve yöneten köprü katmanıdır. Sistem tasarımında yer alan VeritabanıYöneticisi (Singleton) sınıfı bu katmanda çalışarak, ACID prensiplerine uygun veri okuma, yazma ve loglama işlemlerini koordine eder.
- **Veri Katmanı (Data Layer):** Uygulamaya ait kullanıcı bilgilerinin, şifre hash'lerinin, bakiye ve işlem geçmişlerinin kalıcı olarak saklandığı veri katmanıdır. Sistem dışı kapalı bir yapıya sahip olduğundan, veriler bu katmanda yer alan yerel veritabanında güvenli bir şekilde tutulur.



**Figure 1:Sistemin 4 Katmanlı Alt Sistem Ayrıştırması**

## 2.2. Donanım/Yazılım Eşlemesi

Banka Yönetim Sistemi, platform bağımsız bir web veya mobil uygulama yerine, doğrudan masaüstü (desktop) ortamında çalışmak üzere tasarlanmıştır. Sistemin donanım gereksinimleri optimum düzeyde tutulmuş olup, fiziksel bir bankacılık donanımına ihtiyaç duyulmadan tüm süreçler yazılım arayüzü üzerinden simüle edilmektedir.

Yazılım, C++ ve Qt entegrasyonunun sunduğu esneklik ve taşınabilirlik sayesinde modern masaüstü işletim sistemleri (Windows, Linux, macOS vb.) üzerinde tam uyumlulukla derlenip çalışacak şekilde genel bir yapılandırmaya sahiptir.

Geliştirme ve derleme süreçleri, kodun hızla test edilebilmesi amacıyla yüksek konfigürasyonlu sistemlerde yürütülmüş olsa da; bu değerler uygulamanın çalışması için gereken minimum sistem gereksinimlerini yansıtmamaktadır. C++'ın düşük bellek ve kaynak tüketimi avantajı sayesinde derlenmiş uygulama, çok daha mütevazı donanımlara sahip standart ofis veya ev bilgisayarlarında (örneğin 4 GB RAM ve giriş seviyesi işlemciler) akıcı bir şekilde ve performans kaybı yaşanmadan çalışabilmektedir.

Veri yönetimi açısından sistem dış API'lere ve gerçek bankacılık ağlarına kapalı bir simülasyon olarak tasarlandığından, veritabanı uzak bir bulut sunucusunda değil, doğrudan uygulamanın çalıştığı istemci makine (client machine) üzerinde yerel olarak konumlandırılmıştır.

### 2.3. Kalıcı Veri Yönetimi

Proje kapsamında kalıcı olarak saklanacak veri grupları Tablo 1'de, seçilen veritabanı teknolojisi ile bu seçimin teknik gerekçeleri Tablo 2'de özetlenmiş olup; sistemin veritabanı mimarisini modelleyen ER diyagramı ise Şekil 1'de sunulmuştur.

| Veri Kategorisi            | Tablo Adı | Açıklama                                                                                                             |
|----------------------------|-----------|----------------------------------------------------------------------------------------------------------------------|
| <b>Kullanıcı Bilgileri</b> | KULLANICI | Müşteri, personel ve admin kullanıcılarının kimlik bilgileri, iletişim verileri ve şifrelenmiş parola hash değerleri |
| <b>Hesap Bilgileri</b>     | HESAP     | Kullanıcılara ait vadesiz/vadeli hesaplar, bakiye, para birimi ve hesap durumu                                       |
| <b>İşlem Kayıtları</b>     | ISLEM     | Çekme, yatırma ve transfer işlemlerinin tutarı, türü, durumu ve zaman damgası                                        |
| <b>Dekont Bilgileri</b>    | DEKONT    | Başarılı işlemlere ait dekont numaraları ve detay bilgileri (JSON formatında)                                        |
| <b>Kredi Verileri</b>      | KREDI     | Kredi başvuruları, faiz oranları, taksit planları, onay/red durumları ve kredi skorları                              |
| <b>Sistem Logları</b>      | LOG_KAYIT | Kullanıcı giriş/çıkış, işlem ve hata kayıtları ile IP adresi bilgileri                                               |
| <b>Bildirimler</b>         | BILDIRIM  | Kullanıcılara gönderilen SMS, e-posta ve sistem bildirimlerinin içerikleri ve okunma durumları                       |

**Tablo 1:Kalıcı Olarak Saklanan Veriler**

| Kriter                     | Gerekçe                                                                                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Gömülü Mimari</b>       | SQLite, sunucu gerektirmeyen (serverless) bir veritabanı motorudur. Qt/C++ masaüstü uygulamasına doğrudan gömülebilir; ayrı bir veritabanı sunucusu kurulumu ve bakımı gerekmez. |
| <b>Qt Entegrasyonu</b>     | Qt Framework, QSqlDatabase modülü aracılığıyla SQLite için yerleşik (built-in) sürücü desteği sunar. Ek bir kütüphane veya bağımlılık gerektirmeden doğrudan kullanılabilir.     |
| <b>Tek Dosya Yapısı</b>    | Tüm veritabanı tek bir .sqlite dosyasında saklanır. Bu sayede yedekleme, taşıma ve dağıtım işlemleri basitleşir.                                                                 |
| <b>ACID Uyumluluğu</b>     | SQLite, tam ACID (Atomicity, Consistency, Isolation, Durability) desteği sağlar. Bu özellik, finansal işlemlerde veri bütünlüğünün korunması için kritik öneme sahiptir.         |
| <b>Sıfır Konfigürasyon</b> | Kurulum ve yapılandırma gerektirmez. Uygulama çalıştırıldığında veritabanı dosyası otomatik olarak oluşturulur ve kullanıma hazır hale gelir.                                    |
| <b>Performans</b>          | Masaüstü ölçeğindeki uygulamalar için yeterli okuma/yazma performansı sunar. İndeks desteği ile sık sorgulanan alanlarda sorgu süreleri optimize edilmiştir.                     |
| <b>Taşınabilirlik</b>      | Platform bağımsız çalışır (Windows, Linux, macOS). Veritabanı dosyası herhangi bir platformda değişiklik yapılmadan kullanılabilir.                                              |

**Tablo 2: Veritabanı Teknolojisi ve Seçim Gerekçeleri**





**Şekil 1:ER DİYAGRAMI**

## 2.4. Erişim Kontrolü ve Güvenlik

Sistem içerisinde kullanıcı rollerine göre erişim sınırlarını net bir şekilde belirlemek amacıyla bir erişim kontrol matrisi tanımlanmıştır. Bu matris, müşteri, banka personeli ve sistem yöneticisi rollerinin hangi işlemleri gerçekleştirebileceğini açıkça ortaya koyarak hem güvenlik hem de yetki yönetimi açısından sistemin tutarlı bir şekilde çalışmasını sağlamaktadır.

| Kullanım Durumu                         | Müşteri | Banka Personeli | Sistem Yöneticisi |
|-----------------------------------------|---------|-----------------|-------------------|
| Giriş Yap / Çıkış Yap                   | ✓       | ✓               | ✓                 |
| Şifre Sıfırlama                         | ✓       | ✓               | ✓                 |
| Müşteri Profil Yönetimi                 | ✓       | ✓               | ✓                 |
| Yeni Hesap Açma / Kapatma               | ✓       | ✓               | ✓                 |
| Hesap Bakiyesi Görüntüleme              | ✓       | X               | X                 |
| Para Çekme / Yatırma / Transfer         | ✓       | X               | X                 |
| Banka Kartı Oluşturma ve PIN İşlemleri  | ✓       | X               | X                 |
| Dekont Oluşturma                        | ✓       | X               | X                 |
| Müşteri Kaydı Ekleme / Güncelleme       | X       | X               | ✓                 |
| Hesap Dondurma / Silme                  | X       | X               | ✓                 |
| Sistem Loglarını ve Tüm Kayıtları Görme | X       | X               | ✓                 |

**Tablo 3:ERİŞİM KONTROL MATRİSİ**

### **Kimlik Doğrulama:**

- **Kullanıcı Adı ve Şifre Kontrolü:** Her kullanıcı, sisteme benzersiz bir kullanıcı adı/ID ve şifre ile giriş yapar.
- **Şifre Özetleme:** Güvenlik gereği şifreler veri tabanında açık metin olarak değil, C++ tarafında uygulanacak şifreleme algoritmalarıyla özetlenmiş haliyle saklanır.
- **Oturum Yönetimi:** Başarılı giriş sonrası kullanıcı oturumu başlatılır ve çıkış yapıldığında oturum güvenli bir şekilde sonlandırılarak kullanıcı giriş ekranına yönlendirilir.

### **Yetkilendirme:**

- **Kalıtım Yoluyla Yetki Ayrımı:** Kullanıcıların sadece kendi rolleri dahilindeki işlemleri yapabilmesi için nesne yönelimli programlama prensipleriyle birbirlerinden izole edilmişlerdir.
- **Fonksiyonel Kısıtlama:** Arayüz, kullanıcının rolüne göre şekillenir.

### **Veri Güvenliği:**

- **Kapsülleme:** Bakiye ve kullanıcı verileri gizli ve özel olarak tutulur; bu verilere dışarıdan doğrudan erişim engellenerek sadece yetkili metotlar üzerinden işlem yapılması sağlanır.
- **Loglama:** Kritik sistem hareketleri veri tabanına kaydedilir. Bu sayede olası bir güvenlik ihlalinde geçmişe dönük denetim yapılabilir.

## **2.5. Küresel Kontrol Akışı**

Sistemin genel kontrol akışı, kullanıcıdan alınan bir isteğin doğrulanması, yetkilendirilmesi, işlenmesi ve sonuçlandırılması adımlarından oluşmaktadır. Bu süreç, merkezi bir kontrol mekanizması üzerinden yönetilmekte olup tüm işlemler belirli güvenlik ve tutarlılık kontrollerinden geçirilerek yürütülmektedir.

### **2.5.1. Merkezi İş Mantığı (SistemKontrolcüsü)**

Sistem içerisinde kullanıcı arayüzü ile veri katmanı arasındaki tüm etkileşimler, merkezi bir kontrol sınıfı olan SistemKontrolcüsü aracılığıyla yönetilmektedir. Bu sınıf, aktif kullanıcı oturumunu takip etmekte, kullanıcı doğrulama işlemlerini gerçekleştirmekte ve finansal işlemleri ACID prensiplerine uygun şekilde yürütmektedir.

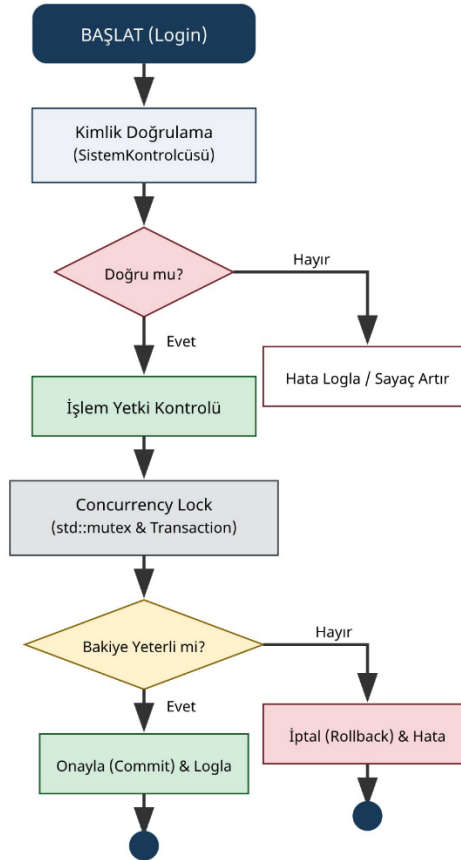
SistemKontrolcüsü sınıfı Singleton tasarım deseni ile gerçekleştirilmiştir. Bu sayede uygulama boyunca yalnızca tek bir kontrol nesnesi kullanılmakta ve

sistem genelinde tutarlı bir kontrol mekanizması sağlanmaktadır. Aynı zamanda veritabanı erişimi de yine tekil bir yapı üzerinden gerçekleştirildiğinden, eşzamanlı işlemler sırasında oluşabilecek veri tutarsızlıkları önlenmektedir.

### 2.5.2. İşlem Yaşam Döngüsü

Sistemde bir işlemin başlangıcından tamamlanmasına kadar geçen süreç belirli adımlar çerçevesinde yürütülmektedir. Kullanıcı sisteme giriş yaptıktan sonra kimlik doğrulama işlemi gerçekleştirilir. Doğrulama başarısız olursa işlem sonlandırılır ve hata loglanır. Başarılı olması durumunda kullanıcı yetkileri kontrol edilir ve işlem gerçekleştirme izni verilir.

İşlem sürecinde, eşzamanlı erişim problemlerini önlemek amacıyla kilitleme mekanizmaları devreye alınır. Ardından bakiye kontrolü yapılır. Eğer bakiye yeterli ise işlem veritabanına yazılır ve commit işlemi gerçekleştirilerek kalıcı hale getirilir. Aksi durumda işlem rollback ile geri alınır ve sistem tutarlı durumuna döndürülür. Bu sürecin görsel temsili ilgili akış şeması ile desteklenmiştir.



Şekil 2:Akış Şeması

### 2.5.3. Güvenlik ve Hata Yönetimi

Sistem, kontrol akışı sırasında oluşabilecek hataları ve güvenlik ihlallerini yönetebilecek şekilde tasarlanmıştır. Başarısız giriş denemeleri sistem tarafından takip edilmekte ve belirli bir eşik değerin aşılması durumunda ilgili hesap güvenlik amacıyla kilitlenmektedir. Kilitlenen hesaplar yalnızca yönetici yetkisine sahip kullanıcılar tarafından tekrar aktif hale getirilebilmektedir.

Ayrıca finansal işlemler sırasında oluşabilecek hata durumlarında, sistem işlemi iptal ederek rollback mekanizması ile önceki tutarlı duruma geri dönmektedir. Tüm hatalar ve kritik işlemler sistem loglarına kaydedilerek izlenebilirlik sağlanmaktadır.

### 2.5.4. Eşzamanlılık ve Veri Tutarlılığı

Sistem, eşzamanlı işlemler sırasında veri tutarlılığını koruyacak şekilde tasarlanmıştır. Bu amaçla iki seviyeli bir koruma mekanizması uygulanmaktadır. İlk olarak, C++ dilinde sunulan std::mutex gibi kitleme mekanizmaları kullanılarak aynı veri üzerinde eşzamanlı işlem yapılması engellenmektedir. İkinci olarak, veritabanı işlemleri ACID prensiplerine uygun olarak yürütülmekte olup, işlemler atomik bir yapı içerisinde gerçekleştirilmektedir.

Bu sayede, işlem sırasında herhangi bir hata oluşması durumunda sistem rollback mekanizması ile önceki duruma döndürülmekte, başarılı işlemler ise commit edilerek kalıcı hale getirilmektedir. Bu yaklaşım, finansal işlemlerde veri bütünlüğünün korunmasını garanti altına almaktadır.

## 2.6. Sınır Koşulları

Sistem, olağandışı durumlar ve işlem süreçlerinin başlangıç ile sonlanma aşamalarını güvenli ve tutarlı bir şekilde yönetebilecek şekilde tasarlanmıştır. Bankacılık işlemlerinin doğası gereği, sistemde yarım kalmış veya tutarsız veri oluşturan işlemlere izin verilmemektedir. Bu nedenle tüm işlemler belirli sınır koşulları çerçevesinde ele alınmaktadır.

### 2.6.1. Başlatma

Bir bankacılık işleminin başlatılması, sistemin tutarlı bir durumdan başka bir tutarlı duruma geçişinin ilk adımını oluşturmaktadır. Bu aşamada temel amaç, işlemin güvenli ve doğru bir şekilde başlatılmasını sağlamaktır.

İşlem başlatılırken öncelikle kullanıcı kimliği ve yetkileri kontrol edilmektedir. Rol tabanlı erişim kontrolü (RBAC) kapsamında, kullanıcının ilgili işlemi gerçekleştirme yetkisine sahip olup olmadığı doğrulanır. Ardından, işleme ait

parametrelerin geçerliliği kontrol edilmekte ve negatif tutar gibi hatalı veri girişleri engellenmektedir.

Bununla birlikte, eşzamanlı işlemlerden kaynaklanabilecek veri tutarsızlıklarını önlemek amacıyla gerekli kaynaklar üzerinde kilitleme mekanizmaları uygulanmaktadır. Bu kilitleme işlemleri, veri yarışlarını (race condition) engelleyerek sistemin tutarlı çalışmasını sağlamaktadır.

## **2.6.2. Sonlandırma**

Bankacılık sistemlerinde işlemler ya tamamen başarılı bir şekilde tamamlanır ya da hiç gerçekleşmemiş gibi iptal edilir. Bu yaklaşım, ACID prensipleri doğrultusunda sistemin veri bütünlüğünü korumak için uygulanmaktadır.

Başarılı bir işlem durumunda (commit), tüm iş kuralları sağlanmış olur ve işlem veritabanına kalıcı olarak yazılır. Bu süreçte geçici veriler diske aktarılır ve işlem sırasında kullanılan kilitler serbest bırakılır.

Herhangi bir aşamada hata oluşması durumunda ise sistem rollback mekanizmasını devreye alır. Bu durumda işlem tamamen iptal edilir ve sistem, işlem öncesindeki tutarlı durumuna geri döndürülür. Özellikle çok adımlı işlemlerde (örneğin para transferi), işlemin bir kısmının başarısız olması durumunda tüm işlem geri alınarak veri tutarsızlığının önüne geçilmektedir.

## **2.6.3. Hata Durumları**

Sistem, farklı türdeki hata senaryolarını yönetebilecek şekilde tasarlanmıştır. Hatalar genel olarak teknik hatalar, iş mantığı hataları, güvenlik hataları ve veri hataları olmak üzere sınıflandırılmaktadır.

Teknik hatalar, genellikle sistem altyapısından kaynaklanan sorunları (örneğin bağlantı zaman aşımı) kapsamakta olup, bu tür durumlarda yeniden deneme (retry) mekanizmaları uygulanmaktadır. İş mantığı hataları, yetersiz bakiye veya limit aşımı gibi durumları içerir ve bu tür hatalarda işlem iptal edilerek kullanıcıya bilgilendirme yapılmaktadır.

Güvenlik hataları kapsamında şüpheli işlemler veya yetkisiz erişim girişimleri tespit edilmekte ve bu durumlarda işlem engellenerek gerekli güvenlik önlemleri alınmaktadır. Veri hatalarında ise hatalı girişler sistem tarafından doğrulama aşamasında tespit edilmekte ve işlemin ilerlemesi engellenmektedir.

## 2.6.4. Belirsiz Durumlar

Dağıtık sistemlerde, işlemin bir kısmının tamamlanmasına rağmen diğer kısmının durumu kesin olarak belirlenemeyebilir. Bu tür durumlar “belirsiz işlem” olarak adlandırılmaktadır.

Bu senaryolarda sistem tutarlılığını korumak amacıyla Two-Phase Commit (2PC) gibi protokoller kullanılmaktadır. Bu yaklaşım sayesinde işlemde yer alan tüm bileşenlerin aynı karar üzerinde uzlaşması sağlanarak veri bütünlüğü korunmaktadır. Ancak bu sistem bir masaüstü simülasyonu olduğu için bu tür senaryolar teorik düzeyde ele alınmıştır.

## 3. Alt Düzey Tasarım

Bu bölümde, Banka Yönetim Sistemi'nin sınıf düzeyindeki tasarımı detaylı bir şekilde ele alınmaktadır. Sistem, nesne yönelimli programlama (OOP) prensipleri doğrultusunda modellenmiş olup; sınıflar, bu sınıfların sorumlulukları, aralarındaki ilişkiler ve sistem içerisindeki etkileşimleri açıklanmaktadır.

Alt düzey tasarım kapsamında; sistemin gerçekleştirilmesi sırasında alınan tasarım kararları, sınıfların organizasyonunu sağlayan paket ve katman yapısı, sınıf arayüzleri ve kullanılan tasarım desenleri incelenmektedir. Bu yaklaşım sayesinde sistemin modülerliği artırılmış, bileşenler arası bağımlılıklar azaltılmış ve yazılımın sürdürülebilirliği ile bakım yapılabilirliği güçlendirilmiştir.

Ayrıca bu bölümde sunulan tasarım, üst düzey mimari kararlar ile uyumlu olacak şekilde kurgulanmış olup, sistemin hem işlevsel gereksinimleri karşılama hem de performans, güvenlik ve veri bütünlüğü gibi kalite hedeflerini desteklemesi amaçlanmıştır.

### 3.1. Nesne Tasarım Kararları

Banka Yönetim Sistemi'nin nesne tabanlı tasarımı sürecinde, projenin temel hedefleri doğrultusunda çeşitli mimari ikilemlerle karşılaşmış ve aşağıdaki kararlar alınmıştır:

- **Güvenlik vs. Kullanılabilirlik:** Sistem bir finansal simülasyon olduğu için veri güvenliği ve yetkisiz erişimlerin engellenmesi en yüksek önceliktir. İşlem güvenliğini maksimize etmek adına her para transferi (havale/virman) ve hesap kapatma işleminde sistemin arka planda yeniden doğrulama yapması ve bakiye kontrollerini katı bir şekilde uygulaması tercih edilmiştir. Sistemde 2 Aşamalı Doğrulama (2FA) gibi karmaşık güvenlik adımları yerine temel kullanıcı adı ve şifre doğrulaması

kullanılarak kullanılabilirlik belirli bir seviyede tutulmaya çalışılmış olsa da, kritik işlemlerde güvenliğin esnetilmemesi yönünde tasarım kararı alınmıştır.

- **Veri Bütünlüğü vs. Geliştirme Hızı:** Projenin analiz raporunda belirtildiği üzere, bakiye güncellemelerinin eşzamanlı gerçekleşmesi ve işlemlerin ACID prensiplerine uygun olarak yürütülmesi kritik bir gereksinimdir. Verileri basit veri yapıları içinde veya metin dosyalarında tutmak geliştirme hızını ciddi oranda artıracak olsa da, finansal simülasyonların doğası gereği veri kaybı veya tutarsızlık yaşanmaması için SQLite tabanlı yerel bir veritabanı mimarisi kurgulanmıştır. Geliştirme süresinin uzaması ve kod karmaşıklığının artması göze alınarak veri bütünlüğü ve sistem doğruluğu tercih edilmiştir.
- **Bakım Kolaylığı vs. Performans:** C++ dili yapısı gereği yüksek performanslıdır. Ancak uygulamanın nesne yönelimli (OOP) prensiplere ve katmanlı mimariye (Model-View-Controller benzeri bir yapı) sıkı sıkıya bağlı kalınarak tasarlanması tercih edilmiştir. Nesneler arası iletişimin soyutlanması, performans açısından küçük de olsa gecikmelere neden olabilir. Ancak ileride eklenebilecek yeni modüllerin sisteme kolayca entegre edilebilmesi ve kodun test edilebilirliği için performans yerine sürdürülebilirlik ve bakım kolaylığı ön planda tutulmuştur.

### **3.2. Son Nesne Tasarımı**

Bu bölümde, Banka Yönetim Sistemi'nin son nesne tasarımı kapsamında sistemde yer alan temel sınıflar, bu sınıfların sorumlulukları ve aralarındaki ilişkiler ele alınmaktadır. Tasarım sürecinde nesne yönelimli programlama (OOP) prensipleri esas alınmış olup, sistemin modüler, sürdürülebilir ve genişletilebilir bir yapıda olması hedeflenmiştir.

Sistem içerisinde yer alan sınıflar, görev ve sorumluluklarına göre gruplandırılarak incelenmiştir. Bu kapsamda, varlık sınıfları (entity), destek sınıfları ve servis/kontrol sınıfları olmak üzere üç ana kategori belirlenmiştir.

#### **3.2.1. Temel Varlık Sınıfları**

Temel varlık sınıfları, sistemdeki ana iş süreçlerini temsil eden ve doğrudan veri taşıyan yapılardır. Kullanıcı, Müşteri, Yönetici, Hesap ve İşlem sınıfları bu gruba dahildir. Bu sınıflar, kullanıcı yönetimi, hesap bilgileri ve finansal işlemler gibi sistemin çekirdek fonksiyonlarını gerçekleştirmektedir.

Bu sınıflar arasında kalıtım ve ilişki yapıları kullanılarak nesneler arası etkileşim sağlanmıştır. Özellikle Kullanıcı sınıfından türeyen Müşteri ve Yönetici sınıfları ile

rol tabanlı bir yapı oluşturulmuştur. Ayrıca Musteri ile Hesap sınıfı arasında bire-çok (one-to-many) ilişkisi kurulmuştur.



### Sınıf Diyagramı - Parça 1: Temel Varlık Sınıfları

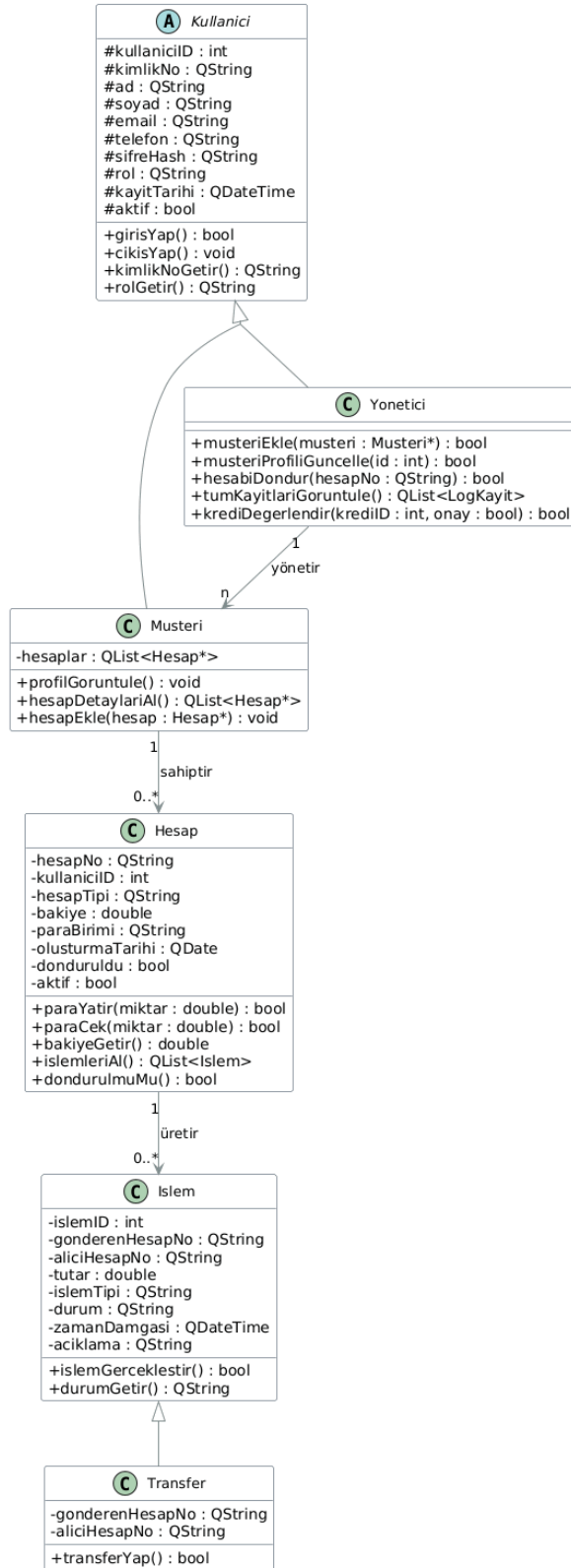


Figure 2: Temel Varlık Sınıfları UML Diyagramı

### 3.2.2. Destek Varlık Sınıfları

Destek varlık sınıfları, sistemin ana iş akışını destekleyen ve ek fonksiyonlar sağlayan yapılardır. Dekont, Kredi, LogKayit ve Bildirim sınıfları bu grupta yer almaktadır.

Dekont sınıfı, gerçekleştirilen işlemlerin çıktılarını temsil ederken, Kredi sınıfı kullanıcıların kredi başvuru ve ödeme süreçlerini modellemektedir. LogKayit sınıfı sistemde gerçekleşen işlemleri ve hataları kayıt altına almakta, Bildirim sınıfı ise kullanıcıya iletilen mesajları temsil etmektedir.

Bu sınıflar, temel varlık sınıfları ile ilişkilendirilerek sistemin bütünsel çalışması sağlanmıştır.

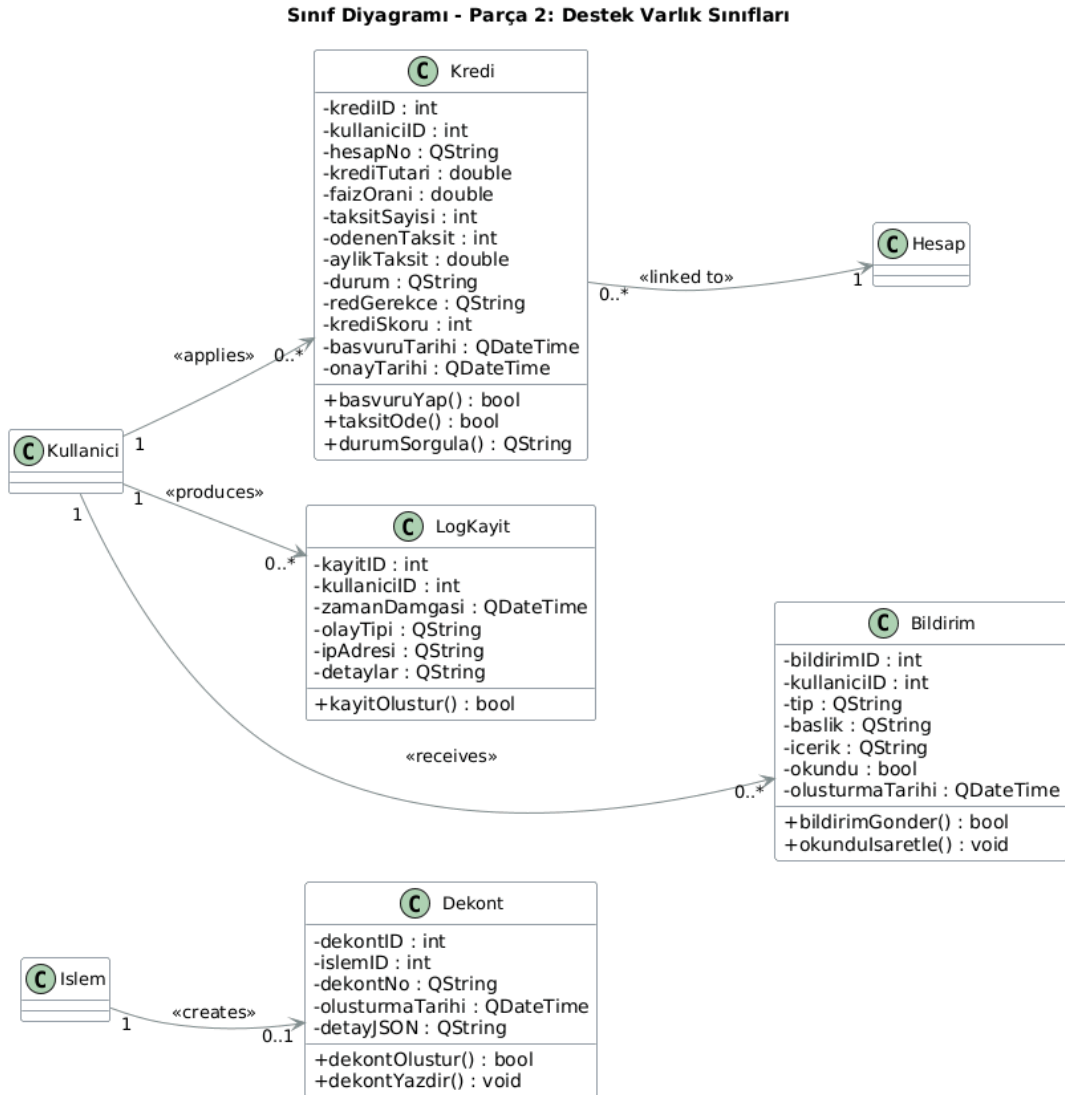


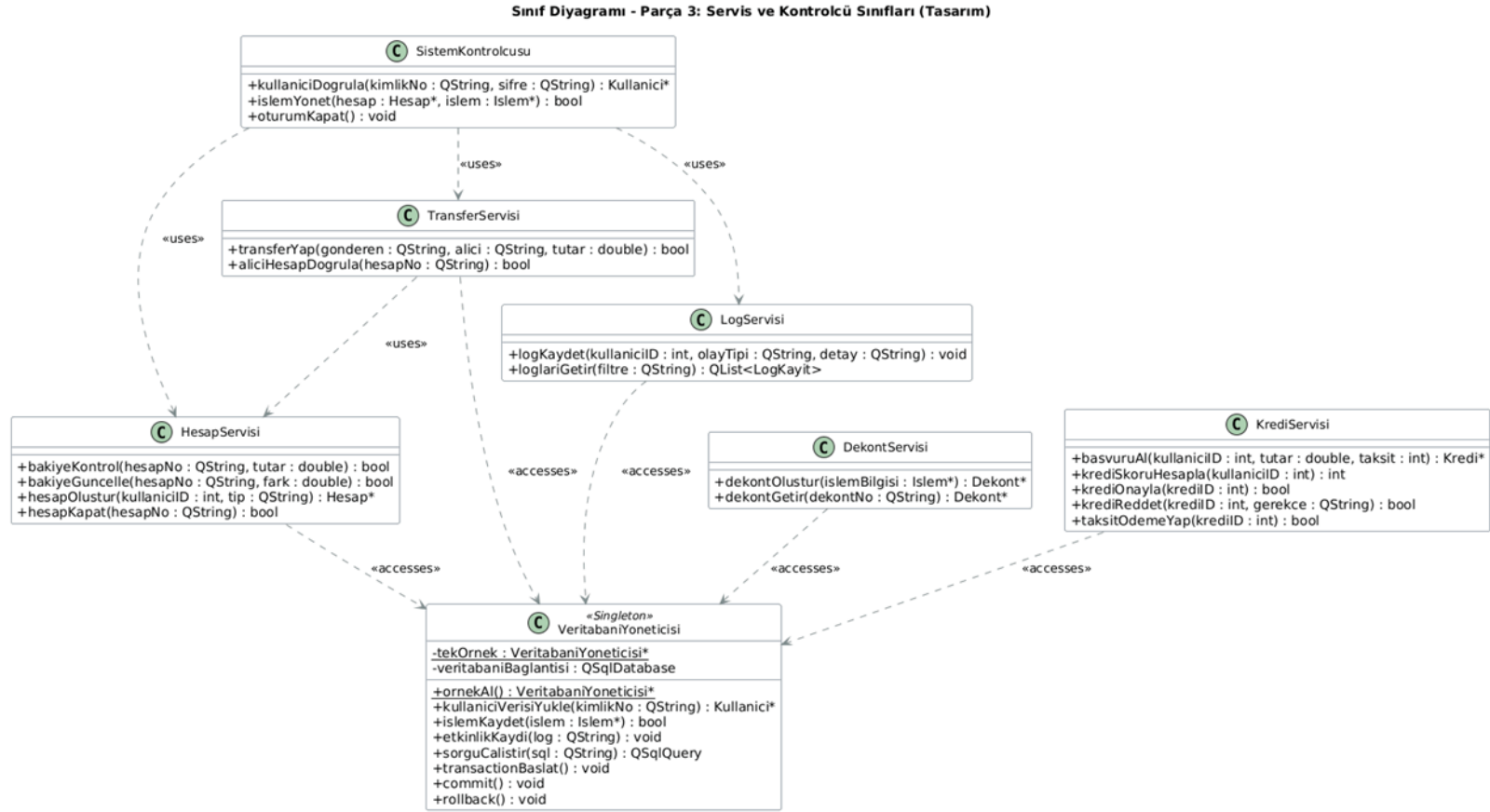
Figure 3: Destek Varlık Sınıfları UML Diyagramı

### 3.2.3. Servis ve Kontrol Katmanı Sınıfları

Servis ve kontrol katmanı sınıfları, sistemin iş mantığını yöneten ve kullanıcı ile veri katmanı arasındaki etkileşimi sağlayan yapılardır. SistemKontrolcusu, HesapServisi, TransferServisi, DekontServisi, KrediServisi ve LogServisi sınıfları bu grupta yer almaktadır.

SistemKontrolcusu sınıfı, kullanıcıdan gelen tüm talepleri yöneten merkezi bir kontrol mekanizması olarak çalışmaktadır. Bu sınıf, gerekli servisleri çağırarak işlemlerin doğru ve güvenli bir şekilde gerçekleştirilmesini sağlar. Servis sınıfları ise belirli işlemlere odaklanarak sistemin modüler yapısını desteklemektedir.

VeritabanıYoneticisi sınıfı ise Singleton tasarım deseni ile yapılandırılmış olup, sistem genelinde tek bir veritabanı erişim noktası sağlamaktadır. Bu yaklaşım, veri tutarlılığını korumak ve eşzamanlı işlemleri güvenli bir şekilde yönetmek açısından önemlidir.



**Figure 4: Servis ve Kontrol Katmanı UML Diyagramı**

Bu yapı sayesinde sistemdeki sınıflar arasında görev ayrımı sağlanmış, bağımlılıklar azaltılmış ve sistemin bakım yapılabilirliği artırılmıştır. Nesneler arası ilişkiler açık bir şekilde modellenerek sistemin anlaşılabilirliği ve genişletilebilirliği desteklenmiştir.

### **3.3. Paketler / Katmanlar**

Banka Yönetim Sistemi tasarlanırken, kodun sürdürülebilirliği, modülerliği ve yeniden kullanılabilirliği ön planda tutulmuştur. Bu doğrultuda katmanlı mimari prensipleri benimsenmiş ve sistem, birbirine gevşek bağlı iç paketler ile platform bağımsızlığı sağlayan dış kütüphanelerin uyumlu bir birleşimi şeklinde yapılandırılmıştır.

#### **3.3.1. Dış Paketler**

Dış paket seçiminde, geliştirme sürecini hızlandıran ve masaüstü sistemlerde kararlı çalışan teknolojilere öncelik verilmiştir. Bu kapsamda, kullanıcı arayüzünün geliştirilmesinde temel çerçeve olarak Qt Framework tercih edilmiştir.

Veritabanı katmanında ise, kapalı devre simülasyon yapısına uygunluğu nedeniyle SQLite kullanılmıştır. Sunucu gerektirmeyen bu yapı sayesinde verilerin yerel ortamda hızlı ve güvenli bir şekilde yönetilmesi sağlanmıştır. Ayrıca veri bütünlüğünün korunması amacıyla ACID prensiplerine uygun bir yaklaşım benimsenmiştir.

Bellek yönetimi ve algoritmik işlemler, C++ Standart Kütüphanesi aracılığıyla gerçekleştirilmiştir.

#### **3.3.2. İç Paketler**

Sistem içerisinde yer alan sınıflar ve modüller, karmaşıklığın azaltılması ve bakım süreçlerinin kolaylaştırılması amacıyla dört ana katman altında toplanmıştır:

##### **Modeller:**

Bu katman, sistemin veri yapısını temsil etmektedir. Kullanıcı, Musteri, Yönetici, Hesap, İşlem, Transfer ve Kayıt gibi sınıflar aracılığıyla veriler nesne yönelimli bir yapı içinde tutulmakta; bakiye, tutar ve tarih gibi kritik finansal bilgiler güvenli bir şekilde saklanmaktadır.

##### **Kontrolcüler ve Servisler:**

Uygulamanın iş mantığı bu katmanda yürütülmektedir. Kullanıcı arayüzü ile veri katmanı arasında bir köprü görevi gören bu yapı içerisinde, kullanıcı doğrulama

ve işlem yönetimi süreçleri gerçekleştirilmektedir. İşlem öncesi gerekli kontroller yapılmakta ve hata durumlarında süreçler uygun şekilde yönetilmektedir.

#### **Veri Erişim Katmanı:**

Veritabanı ile iletişim bu katman üzerinden sağlanmaktadır. VeritabanıYoneticisi sınıfı Singleton tasarım deseni ile yapılandırılmış olup, tüm veri işlemleri merkezi bir yapı üzerinden yürütülmektedir. Arayüzden gelen talepler SQL sorgularına dönüştürülmekte, işlemler kaydedilmekte ve kullanıcı verileri sisteme aktarılmaktadır.

#### **Kullanıcı Arayüzü:**

Kullanıcı etkileşimi bu katman üzerinden sağlanmaktadır. LoginWindow, CustomerDashboard ve AdminPanel gibi arayüz bileşenleri aracılığıyla kullanıcıdan gelen girdiler alınmakta ve kontrol katmanına iletilmektedir. Aynı zamanda sistemdeki veriler kullanıcıya görsel olarak sunulmaktadır.

### **3.4. Sınıf Arayüzleri**

Bu bölümde, sistemde yer alan temel sınıfların dış dünyaya sunduğu arayüzler (public metotlar) ve bu metotların sorumlulukları açıklanmaktadır. Sınıf arayüzleri, sistem bileşenleri arasındaki etkileşimi tanımlamakta ve modüler yapının korunmasını sağlamaktadır.

#### **Kullanici Sınıfı Arayüzü**

Kullanıcıya ait temel kimlik ve oturum işlemlerini yönetir.

- **bool girisYap()**
  - Kullanıcının sisteme giriş yapmasını sağlar.
- **void cikisYap()**
  - Aktif kullanıcı oturumunu sonlandırır.
- **QString kimlikNoGetir()**
  - Kullanıcının kimlik numarasını döndürür.
- **QString rolGetir()**
  - Kullanıcının sistem içindeki rolünü (Müşteri/Yönetici) döndürür.

#### **Hesap Sınıfı Arayüzü**

Kullanıcının finansal işlemlerini yönetir.

- **bool paraYatir(double miktar)**
  - Hesaba para yatırma işlemini gerçekleştirir.
- **bool paraCek(double miktar)**
  - Hesaptan para çekme işlemini gerçekleştirir.
- **double bakiyeGetir()**
  - Hesabın güncel bakiyesini döndürür.
- **bool dondur()**
  - Hesabı pasif duruma getirir.

### **Islem Sınıfı Arayüzü**

Gerçekleştirilen finansal işlemleri temsil eder.

- **bool islemGerceklestir()**
  - İşlemi başlatır ve yürütür.
- **QString durumGetir()**
  - İşlemin durumunu (başarılı/başarısız) döndürür.

### **Transfer Sınıfı Arayüzü**

İki hesap arasında para transferini sağlar.

- **bool transferYap()**
  - Gönderici ve alıcı hesap arasında para transferi gerçekleştirir.

### **SistemKontrolcusu Sınıfı Arayüzü**

Sistemin merkezi kontrol mekanizmasıdır.

- **Kullanici\* girisDogrula(QString id, QString sifre)**
  - Kullanıcı kimlik doğrulamasını yapar ve uygun kullanıcı nesnesini döndürür.
- **bool islemYonet(Hesap\* hesap, Islem\* islem)**
  - Finansal işlemleri kontrol eder ve yürütür.

- **void oturumKapat()**
  - Kullanıcı oturumunu sonlandırır.

### **HesapServisi Sınıfı Arayüzü**

Hesap işlemlerine ait iş mantığını içerir.

- **bool bakiyeKontrol(QString hesapNo, double tutar)**
  - İşlem öncesi bakiye yeterliliğini kontrol eder.
- **bool bakiyeGuncelle(QString hesapNo, double tutar)**
  - Hesap bakiyesini günceller.

### **TransferServisi Sınıfı Arayüzü**

Transfer işlemlerinin yönetimini sağlar.

- **bool transferYap(QString gonderen, QString alici, double tutar)**
  - Hesaplar arası para transferini gerçekleştirir.

### **VeritabanıYoneticisi Sınıfı Arayüzü (Singleton)**

Veritabanı erişimini merkezi olarak yönetir.

- **static VeritabanıYoneticisi\* getInstance()**
  - Tekil veritabanı nesnesini döndürür.
- **bool baglan()**
  - Veritabanı bağlantısını başlatır.
- **bool sorguCalistir(QString sorgu)**
  - SQL sorgularını çalıştırır.
- **void commit()**
  - İşlemi kalıcı hale getirir.
- **void rollback()**
  - Hata durumunda işlemi geri alır.
  -



### 3.5. Tasarım Desenleri

#### 1. Singleton Deseni:

- Neden Tercih Edildi?

Bankacılık simülasyonunda veri tabanı (SQLite) bağlantısının sistem genelinde yalnızca bir kez oluşturulması ve tüm modüllerin bu ortak bağlantı üzerinden işlem yapması gerekmektedir. Birden fazla veri tabanı bağlantısı açmak bellek israfına ve eşzamanlı finansal işlemlerde veri tutarsızlıklarına yol açabilir. Singleton deseni, veri tabanı bağlantısının tek bir noktadan, güvenli bir şekilde yönetilmesini sağlar.

- Uygulandığı Sınıflar

VeritabanıYöneticisi sınıfı.

#### 2. Factory Method Deseni:

- Neden Tercih Edildi?

Sistemde farklı yetkilere sahip kullanıcı tipleri bulunmaktadır. Kullanıcı sisteme giriş yaptığında, arayüzün doğrudan alt sınıfları bilmesine gerek yoktur. Fabrika deseni sayesinde, girilen kimlik bilgilerine göre sistem dinamik olarak doğru kullanıcı tipinden bir nesne üretir.

- Uygulandığı Sınıflar

Kullanici sınıfı, SistemKontrolcüsü sınıfı.

#### 3. Observer Deseni:

- Neden Tercih Edildi?

Arka plandaki finansal veri değiştiğinde, kullanıcı arayüzünün bu değişiklikten anında haberdar olması için Observer deseni kullanılır. Signal/Slot mekanizması, bu desenin bir uygulamasıdır.

- Uygulandığı Sınıflar

Hesap sınıfı, Qt Arayüz sınıfı/sınıfları.

#### 4. Facade Deseni:

- Neden Tercih Edildi?

Sistem birçok karmaşık alt servisten oluşmaktadır. Facade deseni, Qt arayüzünün bu alt sistemlerin karmaşıklığıyla uğraşmasını engeller. Arayüz tüm isteklerini tek bir yönetici sınıfa gönderir ve arka plandaki süreçler bu cephe sınıfı tarafından koordine edilir.

- Uygulandığı Sınıflar

SistemKontrolcüsü sınıfı.

## **Sözlük (Glossary)**

### **ACID (Atomicity, Consistency, Isolation, Durability):**

Veritabanı işlemlerinin güvenli ve tutarlı bir şekilde gerçekleştirilmesini sağlayan dört temel prensiptir. İşlemlerin ya tamamen gerçekleşmesini ya da hiç gerçekleşmemesini garanti eder.

### **Concurrency (Eşzamanlılık):**

Birden fazla işlemin aynı anda çalışabilmesi durumudur. Sistem tasarımında veri tutarlılığını korumak için kontrol edilmesi gerekir.

### **Commit:**

Bir veritabanı işleminin başarılı bir şekilde tamamlanarak kalıcı hale getirilmesidir.

### **Rollback:**

Bir işlem sırasında hata oluşması durumunda, yapılan değişikliklerin geri alınarak sistemin önceki durumuna döndürülmesidir.

### **Encapsulation (Kapsülleme):**

Bir sınıfa ait veri ve metotların dış erişime karşı korunması ve yalnızca belirli arayüzler üzerinden erişilmesini sağlayan OOP prensibidir.

### **Inheritance (Kalıtım):**

Bir sınıfın başka bir sınıfın özelliklerini ve davranışlarını devralmasıdır.

### **Singleton Pattern:**

Bir sınıftan yalnızca bir adet nesne oluşturulmasını sağlayan tasarım desenidir. Bu projede veritabanı yönetimi için kullanılmıştır.

### **MVC (Model-View-Controller):**

Yazılım mimarisinde veri (Model), kullanıcı arayüzü (View) ve iş mantığını (Controller) birbirinden ayıran tasarım yaklaşımıdır.

**Transaction (İşlem):**

Veritabanı üzerinde gerçekleştirilen bir veya daha fazla adım içeren işlemler bütünüdür.

**UML (Unified Modeling Language):**

Yazılım sistemlerini görsel olarak modellemek için kullanılan standart diyagram dilidir.

**GUI (Graphical User Interface):**

Kullanıcının sistemle etkileşime girdiği grafiksel arayüzdür.

**RBAC (Role-Based Access Control):**

Kullanıcıların sistemdeki yetkilerinin rollerine göre belirlendiği erişim kontrol modelidir.

## **Referanslar**

[1] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[2] E. Gamma, R. Helm, R. Johnson and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.